Lab 5, Group A: Activation Function Comparison on MNIST

Anatoli Abahumna Leonardo Meloni

Class group 102, group 16, variant A

Implementation Overview

The multilayer perceptron was implemented from scratch with NumPy. The implementation contains dense layers, activation layers, mean squared error loss, mini-batch stochastic gradient descent, accuracy evaluation, and reusable network construction from an arbitrary list of layer sizes. Each dense layer stores its input during the forward pass and updates its weights and bias during backpropagation. Inputs are processed in batches, so the same code supports both single-hidden-layer and deeper architectures.

Weights were initialized with a Xavier-style uniform range based on the input and output size of each dense layer, while biases were initialized to zero. The training loop shuffles the training set at the start of each epoch, computes the loss on mini-batches, and then propagates the error backward through the layer list in reverse order. No machine learning framework or prebuilt model class was used for the network itself.

MNIST images were scaled to the range $[0, 1]$, and labels were converted to one-hot vectors. The hidden activation function was varied between experiments. The output layer used a sigmoid activation in all runs so that the MSE loss was applied to outputs in the same range as the one-hot targets.

Activation Functions

The compared hidden activations were:

$$\tanh(x), \qquad \sigma(x) = \frac{1}{1 + e^{-x}}, \qquad \mathrm{ReLU}(x) = \max(0, x),$$

$$\mathrm{LeakyReLU}(x) = \begin{cases} x, & x > 0, \\ 0.01x, & x \leq 0. \end{cases}$$

Their derivatives were implemented directly and used during backpropagation.

Experimental Setup

All activation functions used the same settings: architecture 784-64-32-10, MSE loss, SGD, learning rate 0.5, batch size 128, and 18 epochs. Results were averaged over three random seeds: 7, 21, and 42. The experiment used a stratified 80/20 MNIST split, giving 56,000 training images and 14,000 test images.

Accuracy Results

Table 1 reports the mean test accuracy and standard deviation for each activation over the three random seeds.

Table 1: Test accuracy by hidden activation over three seeds.

Activation	Mean test accuracy	Standard deviation
Sigmoid	0.9541	0.0007
Tanh	0.9722	0.0005
ReLU	0.9740	0.0004
LeakyReLU	0.9741	0.0016

The sigmoid network learned noticeably more slowly than the other networks. Tanh reduced the loss much faster, but the final accuracy remained below the rectified activations. ReLU and LeakyReLU produced almost the same result: LeakyReLU had the highest mean accuracy, while ReLU had a slightly smaller standard deviation. The difference between them is small, so the main result is that both rectified activations were stronger than sigmoid and tanh under these settings.

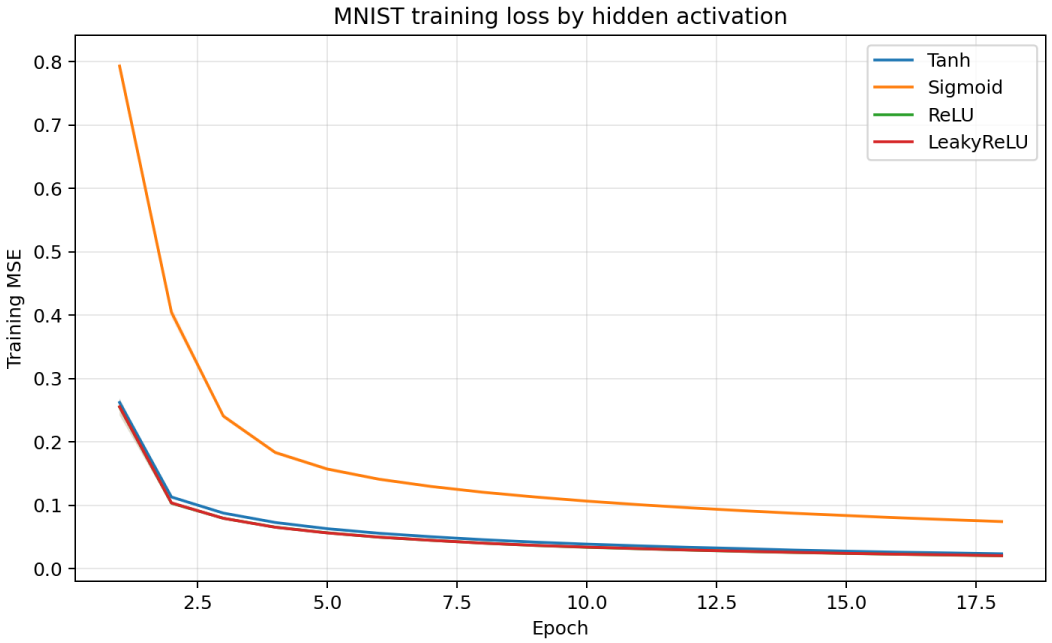


Figure 1: Mean training loss curves over three seeds for each hidden activation.

Architecture Comparison

LeakyReLU had the highest mean accuracy, so it was used for the architecture comparison. The wider architecture improved the mean test accuracy and reached a lower final training loss. The comparison is shown in Table 2.

Table 2: LeakyReLU architecture comparison over three seeds.

Architecture	Mean test accuracy	Standard deviation
784-64-32-10	0.9741	0.0016
784-128-64-10	0.9792	0.0004

The wider network improved the mean accuracy by about half a percentage point and also reduced the variation across seeds. Its lower training loss indicates that the baseline architecture

still had some capacity limitation for this training setup. At the same time, the test accuracy also improved, so the added hidden units did not show harmful overfitting in this experiment.

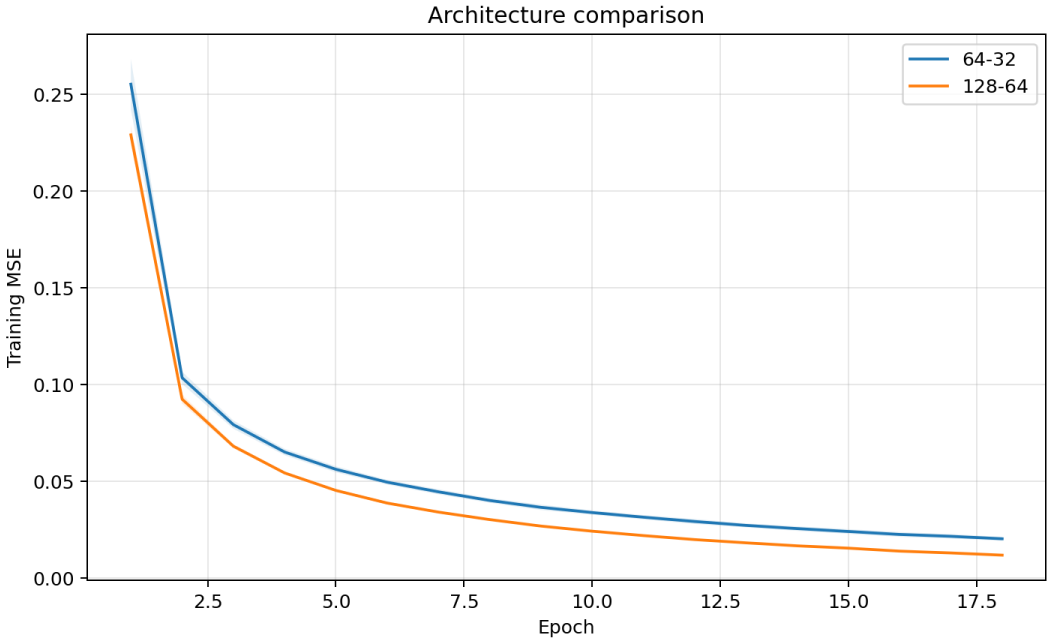


Figure 2: Training loss for the two LeakyReLU architectures.

Conclusions

Sigmoid gave the weakest result and the slowest loss decrease. This is expected because sigmoid saturates for large positive or negative inputs, making its derivative small and increasing the risk of vanishing gradients. Tanh performed better because its outputs are zero-centered, but it can still saturate.

ReLU and LeakyReLU converged faster and reached the best accuracies. Their piecewise linear shape keeps stronger gradients for positive activations, which helps deeper networks train more efficiently. LeakyReLU keeps a small gradient for negative inputs and gave the highest mean accuracy in this setup, although its advantage over ReLU was small. The wider LeakyReLU network performed best overall, suggesting that the additional hidden units gave the model more useful capacity without causing visible instability in these runs.